



JASHORE UNIVERSITY OF SCIENCE AND TECHNOLOGY
Department of Computer Science and Engineering

LAB REPORT

Course Code: CSE 3101

Course Title: Operating System and System Programming Lab

Lab Report No: 10

Lab Title: Implementation of FIFO, LRU, and Optimal Page Replacement Algorithms

Submitted To:

Monishanker Halder,
Assistant Professor,
Dept. of Computer Science and Engineering,
Jashore University of Science and Technology

Submitted By:

Name: Adnan Zaman Niloy
Student ID: 210142
Session: 2021-2022
Year/Semester: 3rd Year, 1st Semester

Date of Submission: 06/08/ 2025

Experiment Title:

Implementation of FIFO, LRU, and Optimal Page Replacement Algorithms

1. Introduction

Page replacement algorithms are essential in virtual memory management, determining which page to remove when physical memory is full. Their choice greatly affects performance, as inefficient replacements increase page faults. Virtual memory enables programs to exceed physical memory limits by swapping pages between main memory and secondary storage. This lab explores three common strategies FIFO, LRU, and Optimal to understand their behavior and efficiency in handling page faults.

2. Objectives

1. **Algorithm Implementation:** To implement and understand the working mechanisms of FIFO, LRU, and Optimal page replacement algorithms using C++ programming language.
2. **Performance Analysis:** To analyze and compare the performance of different page replacement strategies by measuring page fault rates, hit ratios, and miss ratios under identical conditions.
3. **Practical Understanding:** To gain hands-on experience with virtual memory concepts and understand how page replacement decisions affect overall system performance.
4. **Comparative Study:** To evaluate the trade-offs between algorithm complexity and performance efficiency, helping identify suitable algorithms for different scenarios.
5. **Memory Management Insight:** To develop deeper understanding of how operating systems manage limited physical memory resources and handle memory allocation conflicts.

3. Methodology

The experimental methodology follows a systematic approach to implement and test each page replacement algorithm:

3.1 Algorithm Descriptions

FIFO (First-In-First-Out) Algorithm: The FIFO algorithm maintains a queue of pages currently in memory. When a page fault occurs and memory is full, the algorithm removes the page that has been in memory the longest (first in, first out). This approach is simple to implement but doesn't consider page usage patterns, potentially removing frequently accessed pages.

LRU (Least Recently Used) Algorithm: The LRU algorithm tracks the usage history of pages and removes the page that hasn't been accessed for the longest time. This approach assumes that recently used pages are more likely to be used again soon, making it generally more effective than FIFO but requiring additional overhead to track access times.

Optimal Algorithm: The Optimal algorithm, also known as Belady's algorithm, removes the page that will not be used for the longest time in the future. While theoretically optimal in terms of minimizing page faults, it requires knowledge of future page references, making it impractical for real systems but useful as a benchmark for comparison.

3.2 Implementation Strategy

1. **Input Processing:** Read frame size and page reference string from user input
2. **Data Structure Setup:** Initialize appropriate data structures for each algorithm (queues, sets, maps)
3. **Page Reference Simulation:** Process each page reference sequentially
4. **Fault Detection:** Check if the requested page is already in memory
5. **Replacement Logic:** Apply algorithm-specific replacement strategy when memory is full
6. **Performance Metrics:** Calculate and display hit ratio, miss ratio, and total page faults

3.3 Testing Approach

All algorithms are tested with identical input data to ensure fair comparison. The test case uses a frame size of 3 and a page reference string of [0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3], which provides a good mix of repeated and new page references to demonstrate algorithm differences.

4. Source Code

4.1 FIFO Algorithm Implementation

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    int fs, len;
    cin >> fs >> len;
    vector<int> pages(len);
    for (int i = 0; i < len; i++)
        cin >> pages[i];

    queue<int> q;
    int faults = 0;
    set<int> frame;

    for (int page : pages)
    {
        if (frame.find(page) == frame.end())
        {
```

```

        faults++;
        if (frame.size() == fs)
        {
            int removed = q.front();
            q.pop();
            frame.erase(removed);
        }
        frame.insert(page);
        q.push(page);
    }
}

double miss_ratio = (double)faults / len;
double hit_ratio = 1 - miss_ratio;
cout << "Miss Ratio: " << miss_ratio * 100 << "%" << endl;
cout << "Hit Ratio: " << hit_ratio * 100 << "%" << endl;
cout << "Total Page Faults-FIFO: " << faults << endl;
return 0;
}

```

4.2 LRU Algorithm Implementation

```

#include <bits/stdc++.h>
using namespace std;

int main()
{
    int len, fs;
    cin >> fs >> len;

    vector<int> pages(len);
    for (int i = 0; i < len; i++)
        cin >> pages[i];

    set<int> frames;
    map<int, int> last_used;
    int faults = 0;

    for (int i = 0; i < len; ++i)
    {
        int page = pages[i];
        if (frames.find(page) == frames.end())
        {
            faults++;
            if (frames.size() == fs)
            {
                int lru_time = i;
                int lru_page = -1;

```

```

        for (int f : frames)
        {
            if (last_used[f] < lru_time)
            {
                lru_time = last_used[f];
                lru_page = f;
            }
        }
        frames.erase(lru_page);
    }
    frames.insert(page);
}
last_used[page] = i;
}

double miss_ratio = (double)faults / len;
double hit_ratio = 1 - miss_ratio;
cout << "Miss Ratio: " << miss_ratio * 100 << "%" << endl;
cout << "Hit Ratio: " << hit_ratio * 100 << "%" << endl;
cout << "Total Page Faults-LRU: " << faults << endl;
return 0;
}

```

4.3 Optimal Algorithm Implementation

```

#include <bits/stdc++.h>
using namespace std;

int main()
{
    int fs, len;
    cin >> fs >> len;

    vector<int> pages(len);
    for (int i = 0; i < len; i++)
        cin >> pages[i];

    set<int> frames;
    int faults = 0;

    for (int i = 0; i < len; i++)
    {
        int page = pages[i];
        if (frames.find(page) == frames.end())
        {
            faults++;
            if (frames.size() == fs)
            {

```

```

        int farthest = i;
        int victim = -1;

        for (int f : frames)
        {
            int next_use = len;
            for (int j = i + 1; j < len; j++)
            {
                if (pages[j] == f)
                {
                    next_use = j;
                    break;
                }
            }
            if (next_use > farthest)
            {
                farthest = next_use;
                victim = f;
            }
        }
        frames.erase(victim);
    }
    frames.insert(page);
}

double miss_ratio = (double)faults / len;
double hit_ratio = 1 - miss_ratio;
cout << "Miss Ratio: " << miss_ratio * 100 << "%" << endl;
cout << "Hit Ratio: " << hit_ratio * 100 << "%" << endl;
cout << "Total Page Faults-Optimal: " << faults << endl;
return 0;
}

```

5. Results and Discussion

5.1 Test Case Input

- **Frame Size:** 3
- **Page Reference String:** [0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3]
- **Total References:** 11

5.2 Algorithm Performance Results

FIFO Algorithm Results:

- Total Page Faults: 9
- Miss Ratio: 81.82%

- Hit Ratio: 18.18%

LRU Algorithm Results:

- Total Page Faults: 8
- Miss Ratio: 72.73%
- Hit Ratio: 27.27%

Optimal Algorithm Results:

- Total Page Faults: 6
- Miss Ratio: 54.55%
- Hit Ratio: 45.45%

5.3 Algorithm Trace Analysis

FIFO Trace: The FIFO algorithm shows the highest number of page faults because it doesn't consider usage patterns. Pages are replaced purely based on arrival time, leading to situations where frequently accessed pages are removed prematurely.

LRU Trace: LRU performs better than FIFO by considering recent usage patterns. It successfully keeps recently accessed pages in memory, reducing the overall fault rate. However, it still cannot predict future access patterns.

Optimal Trace: The Optimal algorithm achieves the best performance as expected, since it has perfect knowledge of future page references. It serves as the theoretical lower bound for page faults in this scenario.

5.4 Performance Comparison

The results clearly demonstrate the performance hierarchy: Optimal > LRU > FIFO. The Optimal algorithm reduces page faults by 33% compared to FIFO, while LRU provides an 11% improvement over FIFO. This shows that considering usage patterns (LRU) significantly improves performance, though not to the theoretical optimum.

5.5 Practical Implications

In real operating systems, LRU or LRU-approximation algorithms are commonly used because they provide a good balance between performance and implementation complexity. FIFO is sometimes used in simple systems due to its minimal overhead, while Optimal serves as a benchmark for evaluating other algorithms.

6. Time and Space Complexity Analysis

6.1 Time Complexity

- **FIFO:** $O(1)$ per page reference for insertion/deletion operations
- **LRU:** $O(n)$ per page reference due to searching for least recently used page
- **Optimal:** $O(n^2)$ per page reference due to future reference scanning

6.2 Space Complexity

- **FIFO:** $O(f)$ where f is the frame size, for queue and set storage
- **LRU:** $O(f)$ for frames set and usage tracking map
- **Optimal:** $O(f)$ for frames set, minimal additional storage needed

7. Conclusion

This lab implemented and compared FIFO, LRU, and Optimal page replacement algorithms. Results showed that Optimal achieved the lowest page faults, followed by LRU and then FIFO. While Optimal is theoretical, LRU offers the best practical balance, explaining its use in real systems. The study highlights the role of locality of reference in performance and provides a solid understanding of virtual memory management.